

Angular HandsOn 1: Environment Setup Project Structure

HANDS-ON 1 [Beginner]

Environment Setup, Project Structure & First Component

Topics Covered:

Angular CLI & Workspace Setup[Check] Angular Project Structure & Files[Check]
Running & Building Angular Apps[Check] Creating Components[Check]
Angular Module Overview[Check]

In this first hands-on, you will install the Angular CLI, scaffold the Student Course Portal project, understand every generated file and folder, and create your first Angular component from scratch.

SET

UP

```
npm install -g @angular/cli then ng new student-course-portal --routing --style=css then cd student-course-portal && ng serve
```

Task 1: Scaffold and Explore the Angular Project

Goal: Create the Angular workspace and understand what each generated file does.

Steps:

1. Run `ng new student-course-portal --routing --style=css`. Accept defaults for all prompts.
2. Open the project in VS Code. For each of the following files, write a one-line comment in a `notes.txt` file explaining its purpose: `angular.json`, `tsconfig.json`, `tsconfig.app.json`, `package.json`, `src/main.ts`, `src/app/app.module.ts` (or `app.config.ts` in standalone mode), `src/app/app.component.ts`, `src/index.html`.
3. Run `ng serve` and open `http://localhost:4200`. Verify the default Angular welcome page loads.
4. Run `ng build` and inspect the `dist/` folder. Note the generated files ? identify which file contains the compiled application code.
5. In `angular.json`, locate the budgets configuration under `architect > build > configurations > production`. Note the `maximumWarning` and `maximumError` thresholds ? document what these control.

Hint:

- Angular 17+ defaults to standalone components (no NgModules required). Angular 20 continues this pattern. If you see `app.config.ts` instead of `app.module.ts`, your project is using the standalone API ? which is the modern approach.
- `src/main.ts` is the application entry point ? it bootstraps either the `AppModule` or the root component directly.

Expected Outcome: `ng serve` runs without errors. `notes.txt` contains explanations for all 8 files. `ng build` produces a `dist/` folder.

Task 2: Create and Organise Components

Goal: Generate the core page components for the Student Course Portal.

Steps:

Angular HandsOn 1: Environment Setup Project Structure

6. Generate four components using Angular CLI: `ng generate component components/header`, `ng generate component pages/home`, `ng generate component pages/course-list`, `ng generate component pages/student-profile`. Observe the 4 files generated for each: `.ts`, `.html`, `.css`, `.spec.ts`.
7. In HeaderComponent's template (`header.component.html`), add a `<nav>` with the portal name 'Student Course Portal' and placeholder links: Home, Courses, Profile.
8. In HomeComponent's template, add a welcome heading (`<h1>`), a short description paragraph, and a stats row showing three hardcoded values: Courses Available: 12, Enrolled: 3, GPA: 3.8.
9. Add `<app-header>` and `<router-outlet>` to `app.component.html`, replacing the default boilerplate content.
10. Verify `ng serve` still compiles without errors and the header and home page content are visible.

Page | For Digital Nurture 5.0 Participants Only

Digital Nurture 5.0 | Angular (v20.0) | Hands-On Exercise Book

Hint:

- `ng generate component` automatically adds the component to the nearest module's declarations (or registers it as standalone in modern Angular). You do not need to register it manually.
- Each component generated by the CLI follows the same structure: `@Component` decorator with `selector`, `templateUrl`, and `styleUrls`. Understand each property before moving on.

Expected Outcome: Browser shows the header nav and home page welcome content. All 4 components generated without errors.

Page | For Digital Nurture 5.0 Participants Only

Digital Nurture 5.0 | Angular (v20.0) | Hands-On Exercise Book